

CONTROLLING PFEIFFER TURBOPUMPS USING RS485

29 Oct 2021 - tsp

Last update 27 Nov 2021

🕒 25 mins

Disclaimer / Warning: Please note that this page *is in no way associated with Pfeiffer*. Everything here has been discovered while hacking around with their product, reading up stuff found on the net, etc. and might also be totally wrong, only partially complete or inaccurate. It just worked for me.

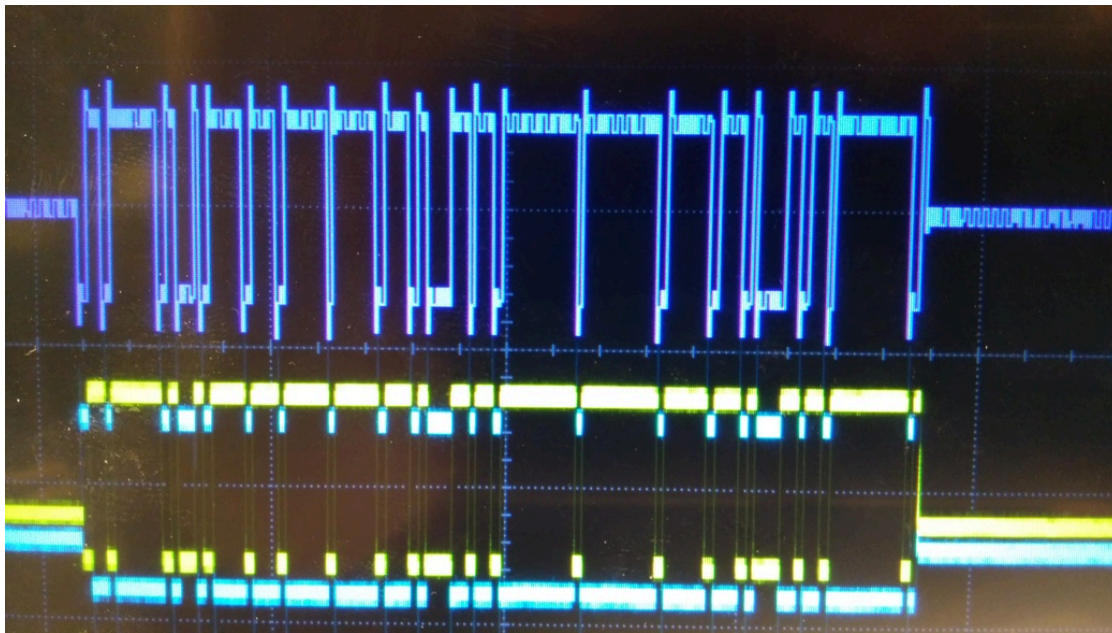


- [Physical layer](#)
 - [Defective TC110](#)
- [Basic data frame format](#)
- [Interfacing the RS485 bus when using the DCU \(sniffing mode\)](#)
 - [Seen protocol deviations from the documentation](#)
 - [First step: Decoding packets using a simply Python script](#)
 - [Second step: An read only MQTT bridge \(work in progress\)](#)
 - [Third step: Implementing a bidirectional command interface \(soft DCU; work in progress\)](#)

PHYSICAL LAYER

The physical layer is pretty simple - it's standard RS485 (technically correct EIA-485). Note this just specifies the electrical interface layer - Pfeiffer does as many other users of EIA-485 bus systems just uses a half duplex solution with a standard USART. As [written earlier when describing the use with AVRs](#) RS485 can cover up to 1.2km range at speeds up to 12Mbps and a maximum of 32 devices per bus (in case their load is 12Ω). Usually voltages between -7V and $+12\text{V}$ are used but the significant part for data transmission is the voltage difference of at least 200mV between the A and B line. Pfeiffer calls the A line D+ and the B line D-.

One can attach any compatible transceiver - though this might of cause void warranty when using something home built as described in this article. The bus is operated at 9600 bits per second with one start and one stop bit. The devices switch between transmitter and receiver mode coordinated by the master device - in most setups that's the *display control unit* (DCU). On more complex setups this might be any vacuum controller or PC.



DEFECTIVE TC110

I've also encountered a defective TC110 turbopump controller. This controller had the A line idle at 0V , rising during transmit to 500mV and the B line idle at 500mV rising to 1V - keeping a constant offset of 500mV . This was also indicated by the display control unit showing a Error E698 while starting up being unable to read out the parameters of the controller. This was most likely caused by a burnt cable during an accident - this seems to have applied 24V directly onto the RS485 lines and thus fried the bus drivers on both ends of the bus system. As soon as we've exchanged the driver ICs on the TC110 as well as the DCU the system worked reliable again.

BASIC DATA FRAME FORMAT

All frames do follow the same format. They consist only of ASCII characters in the range 0x20 to 0x7F inclusive. All frames are separated by an end of frame marker 0x0D (carriage return). The whole protocol is built around a single master / multiple slave architecture. The devices are addressed by a 3 ASCII digit unit address:

- Individual addresses 001 to 255
- Group addresses 9xx for identical units (there won't be any response, only used for commands)
- The global address 000 that matches all units (there won't be any response either)

All frames have the same structure:

Length	Content	Description
3 Byte	a2-a0	This is the device address (see above)
1 Byte	*	Encodes the action to take (see below - 0 basically is a read, 1 a write operation)
1 Byte	0	A single zero always present according to documentation, <i>in reality</i> seems to be always 1
3 Byte	n2-n0	Parameter number. All parameters that can be accessed have a 3 digit ASCII number
2 Byte	l1-l0	ASCII encoded data length in symbols
n Byte	Data	Data payload for the given field
3 Byte	c2-c0	Checksum. Sum of all ASCII octets excluding the checksum field and the end of packet marker encoded as ASCII number modulo 256
1 Byte	0x0D	End of fragment marker

The master supports two different datagram types:

- Something Pfeiffer calls *position request*. This is used to set a parameter or can be interpreted as a command. This basically is a write operation with arbitrary payload size. Action is set to 1.
- A *data request*. This is the read operation and always has data length 02 containing fixed payload =? Action is set to 0.

All slave telegrams follow the same structure. Though there is a positive data request response, a confirmation for a *position request* (write) as well as three different error messages the basic layout is the same as a *position request* including action being set to 1.

- The successful response to a *data request* contains the requested payload
- The successful response to a *position request* contains the data submitted echoed back. This is no confirmation that the write has been successful
- There are three error states that have a payload length of 6 and contain just ASCII error messages:
 - NO_DEF tells one the requested parameter does not exist.
 - _RANGE in case one writes out of range data.

- `_LOGIC` for any logical errors such as writing a read only parameter.

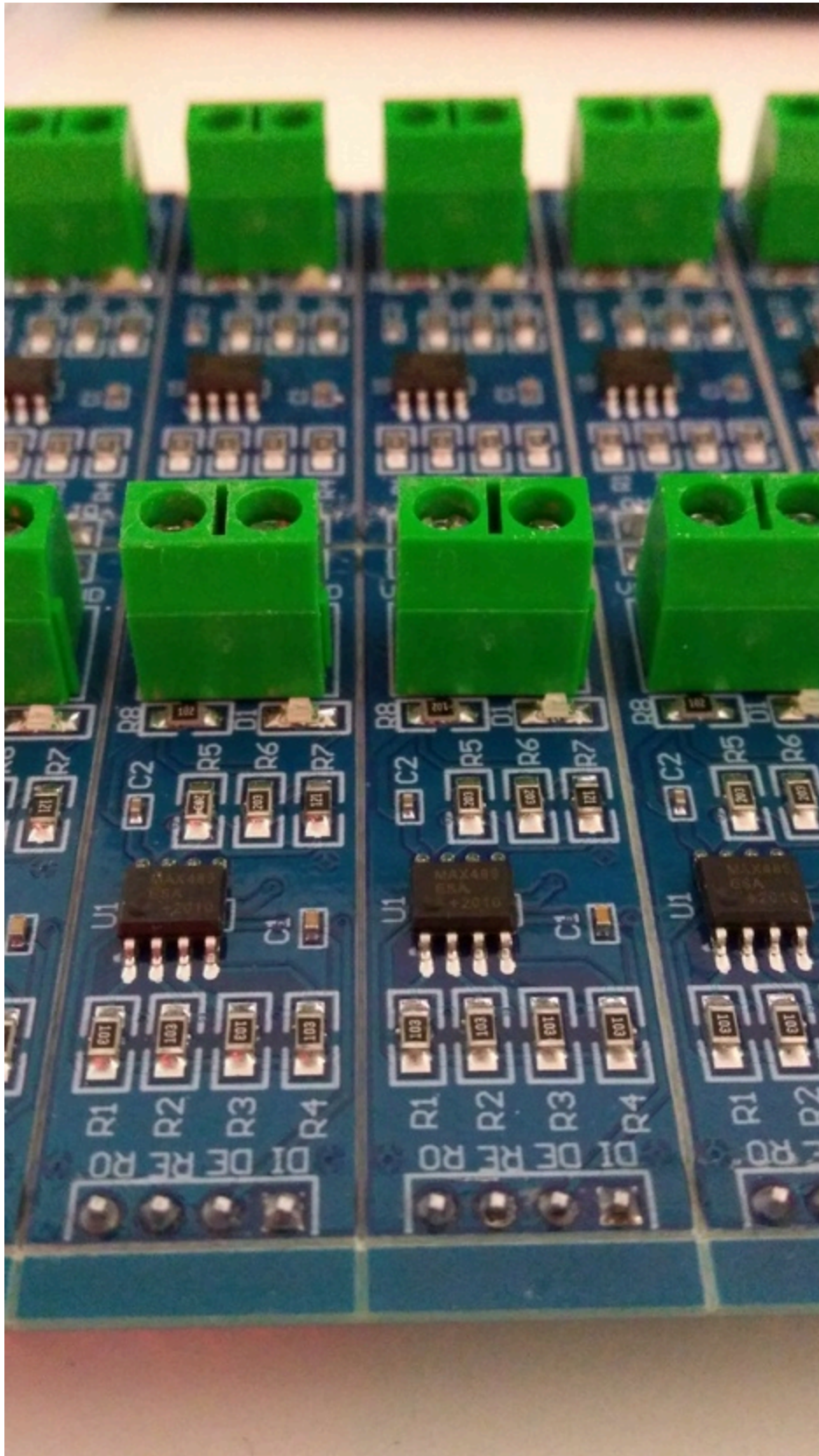
The whole protocol only supports 12 different data types with fixed encodings that are numbered in the manuals:

Datatype	Manual number	Size (symbols)	Description	Sample
boolean_old	0	6	True or false in a somewhat wasteful format (all 1 or 0)	111111 or 000000
u_integer	1	6	Unsigned integer padded with leading zeros, fixed length of 6 characters	012345
u_real	2	6	Fix comma representation, 4 positions in front of and 2 after the comma	123456 equals 1234.56
u_expo	3	6	Positive exponential number including leading zeros	1.2E-6 or 01.2E6
string	4	6	Arbitrary ASCII symbols with values larger or equal to 32	abcdef
vector	5	n	An vector of parameter numbers and parameters. The first two fields are the number of included fields, then always 3 characters parameter number followed by the parameter	02123000000456000
boolean_new	6	1	A new true/false encoding	1 or 0
u_short_int	7	3	Integer number with leading zeros, only 3 characters	012
	8			
tms_old	9	6	Temperature management system control status; 3 boolean (control on / control off) one 3 character u_short integer (temperature)	000037 control off, 37 Celsius; 111457 control on, 457 Celsius
u_expo_new	10	6	Exponential number, first 4 digits are mantissa times 1000, the last two are exponent with offset 20	456711 would be 4.567e-9, 100023 would be 1.000e3
string16	11	16	Any 16 character string with ASCII values larger or equal to 32	abcdefghijklmnp
string8	12	8	Any 8 character string with ASCII values larger or equal to 32	abcdefgh

INTERFACING THE RS485 BUS WHEN USING THE DCU (SNIFFING MODE)

First to make sure I understand the protocol and to understand RS485 operation I decided to only sniff for the messages exchanges between the DCU and the attached turbo- and membrane pumps. To do this I've used a cheap [MAX485 based breakout board](https://www.tspi.at/2021/10/29/pfeifferturbors485.html) attached to an [CP2102 serial to USB adapter](#). Since the RS485 bus is isolated there is no need for any power connection between the devices - even though it's even possible to run without common ground I usually do so ground potential between the sniffer and the other participants on the bus had been connected.

CP2102 Board	RS485 board	Comments
+5V	Vcc	Power supply for the MAX485 is delivered by the CP2102 board
GND	GND	Common ground, should also be attached to the pumps ground
RX	RO	Receive data path
TX	DI	Transmit data path (unused for sniffer)
GND	RE/DE	Tying RE/DE to ground fixes it in receive mode

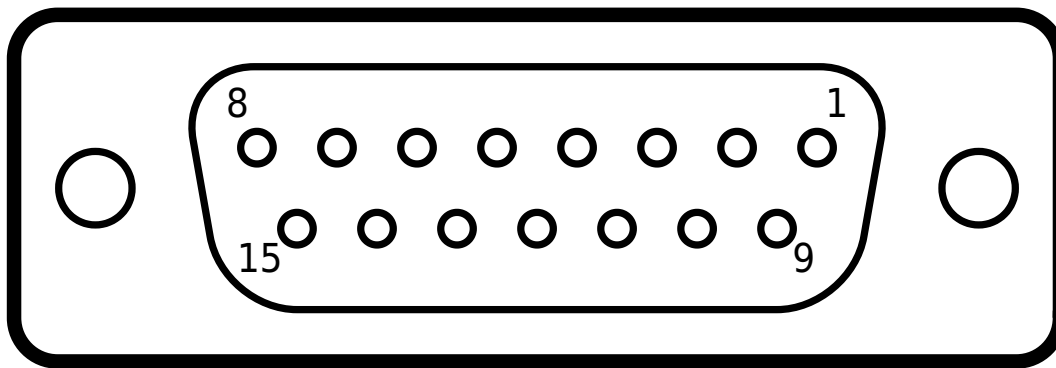


There have been two small modifications that I've made to the board: First RS485 is used in half duplex mode - thus I've tied the DE (driver enable, active high) and RE (receiver enable, active low) pins together. This allows typical [half duplex RS485 operation](https://www.tspi.at/2021/10/29/pfeifferturbors485.html). These pins then have been strapped to ground since only receive operation will be used for sniffing.

The second modification is the bus termination. The breakout boards incorporate a 120Ω bus termination resistor - this should be present on all ends of a typical RS485 bus but not for non-terminal devices since the transmitter simply should see a 120 Ohm impedance when transmitting - and our sniffer will be connected in parallel to the existing devices.

Note that A and B lines on RS485 are *not* interchangeable.

To tap passively into the bus I simply built a simple box using two 15 pin D-Sub connectors (male and female). All pins have been connected 1:1 to allow arbitrary auxiliary signals to be transmitted as if a cable is directly attached. Only the cabling for the RS485 lines and ground has been tapped. The pin assignment on the connector that Pfeiffer calls X3 is:



Pin number	Pin usage	Notes
1	Ub+	Voltage supply for electronic drive unit
2	DI remote priority	Enables or disabled control via X3 interface (open: off, V+: Priority over digital input pins)
3	DI1	Enable venting (Open: off, V+: venting)
4	DI2	Heating (Open: off, V+: heating on)
5	DI pumping station	Open: Off, V+: Error acknowledgment and on
6	DI standby	Standby rotation speed selection (Open: Off, V+: on) and 500-2000 ms pulse error acknowledgment
7	24V DC (V+)	Reference voltage for all digital inputs
8	DO1	Configurable 24V max. 50 mA digital output
9	DO2	Configurable 24V max. 50 mA digital output
10	Accessory A1	
11	Accessory B1	
12	AO1	Rotation speed (0-10V equals 0-100%)
13	RS485 D+	RS485 A
14	RS485 D-	RS485 B
15	Ground	Reference ground

SEEN PROTOCOL DEVIATIONS FROM THE DOCUMENTATION

- Strings (type 4) seem to be allowed an arbitrary number of ASCII characters. They seem not to be limited to 6 chars (I've seen some with a length field of more than 18)
- The fifth byte doesn't seem to be a 0 constant all the time but a 1 constant

FIRST STEP: DECODING PACKETS USING A SIMPLY PYTHON SCRIPT

To get a first feeling I designed a simple bus sniffing utility. This uses the simple wiring described above and a simple Python implementation for the protocol that's able to talk via the serial port. This implementation is available on [GitHub](#) and is also used for a simple monitoring solution (though it turned out to require a little bit more sophisticated approach to determine all desired parameters)

There have been a few differences from the documentation that I encountered while implementing this sniffer:

- The 5'th byte should always be an ASCII 0 according to the documentation. It seems to be always one though
- Strings are documented to be exactly 6 bytes long. They seem to be allowed to have any length

The implementation that I've written allows one to:

- Decode packets captured via the serial port in sniffer mode. These packets can be dumped into a JSON file (not completely valid JSON - one JSON structure per line, no containing array to allow simple append operations) and can also be re-injected in simulation mode. This allows simpler testing of the protocol handlers when one records without specified devices.

The first thing I did was running the sniffer on a pumping station consisting of an DCU 110, a MVP015-4 DC (24V version) membrane backing pump as well as a HiPace 80 turbopump controlled via a TC110 turbopump controller. I've simply attached the tap at the cable connecting the TC110 with the backing pump and the DCU. Then I ran the sniffer in JSON recording mode configuring the two devices (address 1 is the turbopump controller and address 2 the membrane pump):

```
pfeiffersniff -j ./capture.json -d 1:TC110 -d 2:MVP015
```

During startup the DCU queries a bunch of parameters:

```
[DECODED QUERY] 2021-10-19 21:25:25.347298, 1: Name of electronic drive 1  
[DECODED] 2021-10-19 21:25:25.347612, 1: Name of electronic drive unit T
```


[DECODED QUERY] 2021-10-19 21:25:25.347868, 2: Device designation
[DECODED] 2021-10-19 21:25:25.348104, 2: Device designation MVP015
[DECODED QUERY] 2021-10-19 21:25:25.348325, 1: Heating
[DECODED QUERY] 2021-10-19 21:25:25.348533, 1: Standby
[DECODED QUERY] 2021-10-19 21:25:25.348737, 1: Run-up time control
[DECODED QUERY] 2021-10-19 21:25:25.348953, 1: Error acknowledgement
[DECODED QUERY] 2021-10-19 21:25:25.349167, 1: Pumping station
[DECODED QUERY] 2021-10-19 21:25:25.349376, 1: Enable venting
[DECODED QUERY] 2021-10-19 21:25:25.349587, 1: Configuration rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.349810, 1: Configuration output D02
[DECODED QUERY] 2021-10-19 21:25:25.350024, 1: Motor pump
[DECODED QUERY] 2021-10-19 21:25:25.350241, 1: Configuration output D01
[DECODED QUERY] 2021-10-19 21:25:25.350455, 1: Backing pump mode
[DECODED QUERY] 2021-10-19 21:25:25.350661, 1: Rotation speed setting motor
[DECODED QUERY] 2021-10-19 21:25:25.350823, 1: Gas mode
[DECODED QUERY] 2021-10-19 21:25:25.351001, 1: Venting mode
[DECODED QUERY] 2021-10-19 21:25:25.351280, 1: Configuration accessory control
[DECODED QUERY] 2021-10-19 21:25:25.351523, 1: Configuration accessory control
[DECODED QUERY] 2021-10-19 21:25:25.351771, 1: Configuration accessory control
[DECODED QUERY] 2021-10-19 21:25:25.352011, 1: Configuration accessory control
[DECODED QUERY] 2021-10-19 21:25:25.352430, 1: Enable integrated HV sensor
[DECODED QUERY] 2021-10-19 21:25:25.352689, 1: Sealing gas
[DECODED QUERY] 2021-10-19 21:25:25.352910, 1: Configuration output A01
[DECODED QUERY] 2021-10-19 21:25:25.353113, 1: Control via Interface
[DECODED QUERY] 2021-10-19 21:25:25.353315, 1: Interface selection locker
[DECODED QUERY] 2021-10-19 21:25:25.353514, 1: Configuration input DI1
[DECODED QUERY] 2021-10-19 21:25:25.353714, 1: Configuration input DI2
[DECODED QUERY] 2021-10-19 21:25:25.353932, 1: Remote priority
[DECODED QUERY] 2021-10-19 21:25:25.354153, 1: Rotation speed switchpoint
[DECODED QUERY] 2021-10-19 21:25:25.354362, 1: Error code
[DECODED] 2021-10-19 21:25:25.354620, 1: Error code ErrorCode 41
[DECODED QUERY] 2021-10-19 21:25:25.354841, 1: Excess temperature electronic
[DECODED QUERY] 2021-10-19 21:25:25.355002, 1: Excess temperature pump
[DECODED QUERY] 2021-10-19 21:25:25.355127, 1: Set rotation speed attainable
[DECODED QUERY] 2021-10-19 21:25:25.355246, 1: Pump accelerates
[DECODED QUERY] 2021-10-19 21:25:25.355370, 1: Set rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.355507, 1: Active rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.355705, 1: Drive current
[DECODED QUERY] 2021-10-19 21:25:25.355882, 1: Operating hours pump
[DECODED QUERY] 2021-10-19 21:25:25.356009, 1: Firmware version electronic drive
[DECODED] 2021-10-19 21:25:25.356159, 1: Firmware version electronic drive
[DECODED QUERY] 2021-10-19 21:25:25.356379, 1: Drive voltage
[DECODED QUERY] 2021-10-19 21:25:25.356507, 1: Operating hours pump
[DECODED QUERY] 2021-10-19 21:25:25.356629, 1: Nominal rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.356756, 1: Drive power
[DECODED QUERY] 2021-10-19 21:25:25.356967, 1: Pump cycles
[DECODED QUERY] 2021-10-19 21:25:25.357217, 1: Temperature electronic
[DECODED QUERY] 2021-10-19 21:25:25.357471, 1: Temperature pump bottom plate
[DECODED QUERY] 2021-10-19 21:25:25.357691, 1: Acceleration / Deceleration
[DECODED QUERY] 2021-10-19 21:25:25.357860, 1: Temperature bearing
[DECODED QUERY] 2021-10-19 21:25:25.358019, 1: Temperature motor
[DECODED QUERY] 2021-10-19 21:25:25.358176, 1: Name of electronic drive unit
[DECODED] 2021-10-19 21:25:25.358367, 1: Name of electronic drive unit E
[DECODED QUERY] 2021-10-19 21:25:25.358531, 1: Hardware version electronic drive

[DECODED] 2021-10-19 21:25:25.358717, 1: Hardware version electronic dri
[DECODED QUERY] 2021-10-19 21:25:25.358883, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.359069, 1: Error code history, position 1 l
[DECODED QUERY] 2021-10-19 21:25:25.359271, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.359461, 1: Error code history, position 2 l
[DECODED QUERY] 2021-10-19 21:25:25.359623, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.359813, 1: Error code history, position 3 l
[DECODED QUERY] 2021-10-19 21:25:25.360061, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.360252, 1: Error code history, position 4 l
[DECODED QUERY] 2021-10-19 21:25:25.360414, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.360600, 1: Error code history, position 5 l
[DECODED QUERY] 2021-10-19 21:25:25.360765, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.360952, 1: Error code history, position 6 l
[DECODED QUERY] 2021-10-19 21:25:25.361113, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.361298, 1: Error code history, position 7 l
[DECODED QUERY] 2021-10-19 21:25:25.361459, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.361654, 1: Error code history, position 8 l
[DECODED QUERY] 2021-10-19 21:25:25.361908, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.362149, 1: Error code history, position 9 l
[DECODED QUERY] 2021-10-19 21:25:25.362328, 1: Error code history, posit:
[DECODED] 2021-10-19 21:25:25.362515, 1: Error code history, position 10
[DECODED QUERY] 2021-10-19 21:25:25.362676, 1: Set rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.362840, 1: Actual rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.363000, 1: Nominal rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.363157, 1: Set value run-up time
[DECODED QUERY] 2021-10-19 21:25:25.363417, 1: Rotation speed switchpoin
[DECODED QUERY] 2021-10-19 21:25:25.363579, 1: Set value in rotation spe
[DECODED QUERY] 2021-10-19 21:25:25.363737, 1: Set value power consumptio
[DECODED QUERY] 2021-10-19 21:25:25.363901, 1: Switching off threshold fo
[DECODED QUERY] 2021-10-19 21:25:25.364059, 1: Switching on threshold fo
[DECODED QUERY] 2021-10-19 21:25:25.364220, 1: Set value rotation speed
[DECODED QUERY] 2021-10-19 21:25:25.364372, 1: Rotation speed switchpoin
[DECODED QUERY] 2021-10-19 21:25:25.364496, 1: Venting rotation speed at
[DECODED QUERY] 2021-10-19 21:25:25.364616, 1: Venting time at delayed v
[DECODED QUERY] 2021-10-19 21:25:25.364745, 1: Pressure switchpoint 1
[DECODED QUERY] 2021-10-19 21:25:25.364872, 1: Pressure switchpoint 2
[DECODED QUERY] 2021-10-19 21:25:25.364990, 1: Pressure sensor 1 name
[DECODED] 2021-10-19 21:25:25.365136, 1: Pressure sensor 1 name Press1Nar
[DECODED QUERY] 2021-10-19 21:25:25.365259, 1: Pressure value 1
[DECODED QUERY] 2021-10-19 21:25:25.365379, 1: Pressure correction facto
[DECODED QUERY] 2021-10-19 21:25:25.365497, 1: Pressure sensor 2 name
[DECODED] 2021-10-19 21:25:25.365640, 1: Pressure sensor 2 name Press2Nar
[DECODED QUERY] 2021-10-19 21:25:25.365769, 1: Pressure value 2
[DECODED QUERY] 2021-10-19 21:25:25.365889, 1: Pressure correction facto
[DECODED QUERY] 2021-10-19 21:25:25.366008, 1: Nomial rotation speed con
[DECODED QUERY] 2021-10-19 21:25:25.366129, 1: RS-485 device address
[DECODED QUERY] 2021-10-19 21:25:25.366321, 2: Standby
[DECODED QUERY] 2021-10-19 21:25:25.366444, 2: Fault acknowledgement
[DECODED QUERY] 2021-10-19 21:25:25.366561, 2: Pump
[DECODED QUERY] 2021-10-19 21:25:25.366678, 2: Configuration output D02
[DECODED QUERY] 2021-10-19 21:25:25.366802, 2: Configuration output D01
[DECODED QUERY] 2021-10-19 21:26:33.701059, 2: Speed setting mode
[DECODED QUERY] 2021-10-19 21:26:33.701296, 2: Purge gas configuration
[DECODED QUERY] 2021-10-19 21:26:33.701483, 2: Purge gas

```

[DECODED QUERY] 2021-10-19 21:26:33.701666, 2: Control via interface
[DECODED QUERY] 2021-10-19 21:26:33.701855, 2: Interface selection locker
[DECODED QUERY] 2021-10-19 21:26:33.702040, 2: Error code
[DECODED] 2021-10-19 21:26:33.702283, 2: Error code Error code 41
[DECODED QUERY] 2021-10-19 21:26:33.702477, 2: Actual speed
[DECODED QUERY] 2021-10-19 21:26:33.702662, 2: Drive current
[DECODED QUERY] 2021-10-19 21:26:33.702856, 2: Pump operating hours
[DECODED QUERY] 2021-10-19 21:26:33.703040, 2: Software version of the i
[DECODED] 2021-10-19 21:26:33.703274, 2: Software version of the interfa
[DECODED QUERY] 2021-10-19 21:26:33.703494, 2: Supply voltage
[DECODED QUERY] 2021-10-19 21:26:33.703706, 2: Electronic drive unit ope
[DECODED QUERY] 2021-10-19 21:26:33.703925, 2: Nominal speed
[DECODED QUERY] 2021-10-19 21:26:33.704120, 2: Drive power
[DECODED QUERY] 2021-10-19 21:26:33.704304, 2: Temperature of pump
[DECODED QUERY] 2021-10-19 21:26:33.704634, 2: Device designation
[DECODED] 2021-10-19 21:26:33.704910, 2: Device designation ElecName
[DECODED QUERY] 2021-10-19 21:26:33.705140, 2: Actual speed
[DECODED QUERY] 2021-10-19 21:26:33.705363, 2: Hardware version of the i
[DECODED] 2021-10-19 21:26:33.705628, 2: Hardware version of the interfa
[DECODED QUERY] 2021-10-19 21:26:33.705863, 2: Nominal speed
[DECODED QUERY] 2021-10-19 21:26:33.706056, 2: Setpoint in speed setting
[DECODED QUERY] 2021-10-19 21:26:33.706252, 2: Setpoint speed in standby
[DECODED QUERY] 2021-10-19 21:26:33.706456, 2: Setting for purge gas act:
[DECODED QUERY] 2021-10-19 21:26:33.706656, 2: RS485 interface address
[DECODED QUERY] 2021-10-19 21:26:33.706870, 1: Name of electronic drive i
[DECODED] 2021-10-19 21:26:33.707106, 1: Name of electronic drive unit T
[DECODED QUERY] 2021-10-19 21:26:33.707326, 1: Error code
[DECODED] 2021-10-19 21:26:33.707547, 1: Error code 000000
[DECODED QUERY] 2021-10-19 21:26:33.707788, 1: Pump accelerates
[DECODED] 2021-10-19 21:26:33.708032, 1: Pump accelerates False
[DECODED QUERY] 2021-10-19 21:26:33.708284, 1: Heating
[DECODED] 2021-10-19 21:26:33.708516, 1: Heating False
[DECODED QUERY] 2021-10-19 21:26:33.708752, 1: Standby
[DECODED] 2021-10-19 21:26:33.709009, 1: Standby False
[DECODED QUERY] 2021-10-19 21:26:33.709222, 1: Remote priority
[DECODED] 2021-10-19 21:26:33.709390, 1: Remote priority False
[DECODED QUERY] 2021-10-19 21:26:33.709632, 1: Rotation speed switchpoin
[DECODED] 2021-10-19 21:26:33.709804, 1: Rotation speed switchpoint atta
[DECODED QUERY] 2021-10-19 21:26:33.709967, 1: Excess temperature electri
[DECODED] 2021-10-19 21:26:33.710130, 1: Excess temperature electronic d
[DECODED QUERY] 2021-10-19 21:26:33.710293, 1: Set rotation speed attaini
[DECODED] 2021-10-19 21:26:33.710459, 1: Set rotation speed attained Fal
[DECODED QUERY] 2021-10-19 21:26:33.710620, 1: Excess temperature pump
[DECODED] 2021-10-19 21:26:33.710790, 1: Excess temperature pump False
[DECODED QUERY] 2021-10-19 21:26:33.710953, 1: Active rotation speed
[DECODED] 2021-10-19 21:26:33.711131, 1: Active rotation speed 0 Hz
[DECODED QUERY] 2021-10-19 21:26:33.711290, 1: Pumping station

```

As one can see in the beginning I had some problems that not all responses have been correctly captured.

After initialization the DCU in it's default state just queries a fixed parameter set including:

- Is heating enabled?
- Is the turbopump in standby state?
- Has remote control priority?
- Is the rotation switch point attained?
- Is there excess temperature at the electronic drive unit?
- Is the set rotation speed attained?
- Is there excess temperature at the pump?
- What is the current active rotation speed?
- Is the pumping station enabled?

```
[DECODED QUERY] 2021-10-19 21:27:32.199993, 1: Heating
[DECODED] 2021-10-19 21:27:32.200242, 1: Heating False
[DECODED QUERY] 2021-10-19 21:27:32.200612, 1: Standby
[DECODED] 2021-10-19 21:27:32.200839, 1: Standby False
[DECODED QUERY] 2021-10-19 21:27:32.201010, 1: Remote priority
[DECODED] 2021-10-19 21:27:32.201174, 1: Remote priority False
[DECODED QUERY] 2021-10-19 21:27:32.201335, 1: Rotation speed switchpoint
[DECODED] 2021-10-19 21:27:32.201497, 1: Rotation speed switchpoint atta
[DECODED QUERY] 2021-10-19 21:27:32.201662, 1: Excess temperature electri
[DECODED] 2021-10-19 21:27:32.201864, 1: Excess temperature electronic d
[DECODED QUERY] 2021-10-19 21:27:32.202124, 1: Set rotation speed attain
[DECODED] 2021-10-19 21:27:32.202382, 1: Set rotation speed attained Fal
[DECODED QUERY] 2021-10-19 21:27:32.202625, 1: Excess temperature pump
[DECODED] 2021-10-19 21:27:32.202852, 1: Excess temperature pump False
[DECODED QUERY] 2021-10-19 21:27:32.203018, 1: Active rotation speed
[DECODED] 2021-10-19 21:27:32.203194, 1: Active rotation speed 0 Hz
[DECODED QUERY] 2021-10-19 21:27:32.203353, 1: Pumping station
[DECODED] 2021-10-19 21:27:32.203515, 1: Pumping station False
```

It seems that the DCU only changes this in case one selects another parameter on the display to display in live mode. When one then enabled the pump it starts with setting PumpgStatn to True:

```
[DECODED] 2021-10-19 21:32:13.181271, 1: Pumping station True
```

As expected Pump accelerates immediately reads as True and the value of the active rotation speed increases. To turn off during the acceleration phase one simply can set PumpgStatn to False again. It seems there are no commands towards the membrane pump sent by the DCU during startup and shutdown - this might of course also be a problem with the decoding of the messages.

SECOND STEP: AN READ ONLY MQTT BRIDGE (WORK IN PROGRESS)

The next step was to build a simple read only bridge that passes raw messages as well as decoded ones into an MQTT broker. This is a nice solution to modularize the slow control system with a microservice approach. This of course means more services are running and does not offer fast real time guarantees - but it allows one to:

- Modularize the system more - even allows mixing different programming languages and paradigms
- Loose coupling of systems - this minimizes inter component dependencies and reduces the surface for problems.
- Single components are much easier to debug and maintain as well as simpler to deploy.
- Even though there are more services running microservices do scale better
- Easier debugging and simulations
- Higher service reliability than monolithic ones
- Many services are stateless and thus easier to upgrade in a seamless fashion
- Message brokers support caching messages matching subscriptions in persistent sessions even when applications go offline which might be interesting for batch processing.
- Service can be simply composed in an arbitrary way
- There can be an arbitrary number of subscribers for each topic that changes on the fly depending on current requirements. Thus one can decouple logging from control, etc.
- Many IOT devices do support MQTT directly - especially newer developments - and thus can be simply embedded directly into the framework (this gap will be closed with the bridge for the devices)

Of course there are drawbacks:

- Requires an MQTT broker (like [RabbitMQ](#) which is technically an AMQP broker that also supports MQTT) running and configured all the time.
- A little bit more complexity to document and on the first glance it's harder to hack around
- Not usable for realtime stuff or fast control systems. Thus the term slow control like temperature control, vacuum system control, etc. above.
- Need to monitor and instrument the microservices and provide an automated way of managing them if one really wants to work efficiently (on the other hand this is also required for non microservice architectures)

Since the library above had been implemented in Python, the main users of my library use Python everywhere and there is the excellent [Eclipse Paho MQTT](#) client available for Python I decided to build on the protocol library described above.

The mapping between MQTT topics and the pump addresses will be done by assigning each RS485 address a register set which corresponds to the device type as well as an MQTT topic prefix. All messages can also be published under a raw prefix that can be chosen independently. A set of messages is distributed under the prefix that's assigned to each device type - one can also

define that commands can be written to the command endpoint. Having a separate topic endpoint for commands allows one to use the brokers access control.

This is currently work in progress

THIRD STEP: IMPLEMENTING A BIDIRECTIONAL COMMAND INTERFACE (SOFT DCU; WORK IN PROGRESS)

The last step is then of course to implement functionality to control the pumping system - i.e. being able to write registers and actively querying data. This is also work in progress though it won't be hard using the previously mentioned protocol implementation.
